GUÍA PRÁCTICA FORMATIVA: ESTRATEGIA DE DESPLIEGUE BLUE-GREEN (ZERO-DOWNTIME)

Ingeniería de Software V — Sesión 21 (Semana 11)

Enrutamiento Atómico en Nginx, Pre-Calentamiento, Health Checks y Rollback Instantáneo

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS | **DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS INTELIGENTES**

INFORMACIÓN DE LA ACTIVIDAD — IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	**PERÍODO:**	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	**PONDERACIÓN:**	**Formativa** (Sin calificación sumativa) • Duración: 2 Horas
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	**MODALIDAD:**	Parejas / Laboratorio Práctico Guiado
ESTUDIANTE 1:	_____	**CÓDIGO:**	_____
ESTUDIANTE 2:	_____	**CÓDIGO:**	_____

1 Contexto del Problema & Arquitectura Blue-Green

1.1 Descripción del Reto de Despliegue

Tradicionalmente, actualizar una aplicación implicaba detener el proceso anterior, compilar o desplegar el nuevo binario y reiniciar el servicio. Este enfoque destructivo (*In-Place / Recreate*) ocasiona tiempos de caída inevitables (*Downtime*), desconexión de usuarios y errores HTTP 502 Bad Gateway.

La estrategia **Blue-Green Deployment** resuelve este problema manteniendo dos entornos de producción arquitectónicamente idénticos:

- **Entorno Blue (Producción Activa):** Aloja la versión actual (`v1.0.0`) y recibe el 100 % del tráfico real a través del router de entrada.
- **Entorno Green (Entorno Pasivo / Staging):** Aloja la nueva versión (`v2.0.0`). Permite realizar pre-calentamiento de cachés, pruebas de humo y validación de endpoints de salud sin alterar a los clientes.
- **Conmutación Atómica de Tráfico:** Una vez validado Green, el enrutador (Nginx) redirige el 100 % de las conexiones a Green en milisegundos mediante una recarga en caliente (`nginx -s reload`) sin cortar peticiones activas.
- **Rollback Instantáneo:** Si la nueva versión presenta anomalías imprevistas, el router conmutará de vuelta a Blue en menos de 1 segundo.

Componente	Tecnología & Contenedor	Puerto / Endpoint
Router / Gateway	Nginx 1.25 Alpine (Reverse Proxy)	Puerto `8080` (`http://localhost:8080`)
Archivo Auxiliar	`conf.d/upstream.conf` (Dinámico)	Inclusión modular de upstream
Servicio Blue	Microservicio v1.0.0 (`app-blue`)	Puerto interno `8081` (`/api/info`)
Servicio Green	Microservicio v2.0.0 (`app-green`)	Puerto interno `8082` (`/api/info`)
Orquestación	Docker Compose (Red interna `deploy-net`)	Aislamiento y persistencia

2 Procedimiento Experimental Paso a Paso

2.1 Fase 1: Puesta en Marcha del Entorno con Docker Compose (15 min)

Ubíquese en el directorio `codigo_base/` e inspeccione el archivo `docker-compose.yml`:

```
services:
  nginx-router:
    image: nginx:1.25-alpine
    container_name: nginx-router
    ports:
      - "8080:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/conf.d:/etc/nginx/conf.d
    depends_on:
      - app-blue
      - app-green

  app-blue:
    build: ./app-blue
    container_name: app-blue
    ports:
      - "8081:8080" # Puerto expuesto para verificacion y smoke test

  app-green:
    build: ./app-green
    container_name: app-green
    ports:
      - "8082:8080" # Puerto expuesto para verificacion y smoke test
```

Inicie los contenedores y compruebe su estado:

```
docker compose up -d --build
docker compose ps
```

2.2 Fase 2: Arquitectura Modular Nginx y Archivo Auxiliar (15 min)

Fundamento de Ingeniería: Para evitar errores de inodo bloqueado (*Device or resource busy*) al actualizar archivos montados en Docker, Nginx delega la directiva `upstream` a un archivo auxiliar montado dentro del directorio `conf.d`:

```
# nginx/nginx.conf: Incluye la definicion del upstream dinamicamente
http {
    include /etc/nginx/conf.d/upstream.conf;
    server {
        listen 80;
        location / {
            proxy_pass http://backend_app;
            proxy_set_header Host $host;
            add_header X-Handled-By $upstream_addr always;
        }
    }
}

# nginx/conf.d/upstream.conf: Archivo auxiliar (inicialmente apunta a Blue)
upstream backend_app {
    server app-blue:8080;
}
```

Consulte el router principal a través de `curl`:

```
curl -i http://localhost:8080/api/info
```

Debe observar la cabecera `X-Handled-By: ...` y la respuesta activa desde el entorno `BLUE` (versión `1.0.0`).

2.3 Fase 3: Verificación Previa y Pre-Calentamiento en Green (20 min)

Antes de transferir a los usuarios, la versión `v2.0.0` desplegada en Green debe ser sometida a un Smoke Test:

```
# Comprobar salud del microservicio Green antes del swap:
curl -f http://localhost:8082/api/health
curl -s http://localhost:8082/api/info | jq .
```

Si el endpoint no responde HTTP 200 con estado `UP`, la conmutación se cancela inmediatamente, protegiendo a los usuarios.

2.4 Fase 4: Ejecución del Switch Atómico con Tráfico Sintético (30 min)

Para evidenciar que la recarga de Nginx ocurre con cero tiempo de inactividad, abra una terminal y ejecute el generador de tráfico en bucle:

```
# Terminal 1: Generador continuo de peticiones
./test_traffic.sh
```

En una segunda terminal, ejecute el script de conmutación atómica `./switch.sh`:

```
#!/usr/bin/env bash
# 1. Comprobacion de salud estricta en Green
echo "[1/3] Verificando salud de entorno Green..."
curl -sf http://localhost:8082/api/health || { echo "[ERROR] Green no saludable"; exit 1; }

# 2. Modificacion atomica del archivo auxiliar (sin bloquear inodos)
echo "[2/3] Actualizando archivo auxiliar de upstream hacia app-green..."
sed -i 's/app-blue:8080/app-green:8080/' ./nginx/conf.d/upstream.conf

# 3. Recarga atomica en caliente sin cortar conexiones existentes
echo "[3/3] Recargando Nginx en caliente (Zero Downtime)..."
docker exec nginx-router nginx -s reload
echo "[OK] Conmutacion completada. Produccion ahora corre sobre GREEN (v2.0.0)"
```

Observe la Terminal 1: las peticiones cambian de `BLUE` a `GREEN` instantáneamente sin que ninguna arroje error HTTP 502 ni tiempo de espera excesivo.

2.5 Fase 5: Simulación de Incidente y Rollback Instantáneo (20 min)

Simule un fallo en caliente ejecutando el script de reversión:

```
./rollback.sh
```

El script revierte el archivo auxiliar `upstream.conf` hacia `app-blue:8080` y recarga Nginx en menos de 1 segundo.

3 Cuestionario de Reflexión Conceptual & Checklist Formativo

3.1 Preguntas de Reflexión Técnica

1. **Costo de Infraestructura:** ¿Qué implicaciones presupuestales tiene la duplicación al 100 % de recursos que exige Blue-Green en clústeres de cientos de microservicios? ¿Cómo mitiga la nube elástica este costo?

2. **Persistencia de Sesiones:** Si los usuarios almacenan carritos de compra o sesiones en la memoria RAM del contenedor Blue, ¿qué ocurriría al conmutar a Green? ¿Por qué la arquitectura desacoplada Twelve-Factor (Factor VI) exige sesiones en servicios externos (ej. Redis)?

3. **Patrón Expand & Contract en BD:** Si la versión Green requiere una columna nueva en la base de datos relacional compartida, ¿por qué no se debe renombrar ni eliminar una columna previa mientras Blue continúe activo?

4. **Arquitectura de Recarga e Inodos en Docker:** ¿Por qué ejecutar `sed -i` sobre un archivo montado puntualmente (*single file bind mount*) genera el error *Device or resource busy*? ¿Cómo resuelve este problema el patrón arquitectónico de archivo auxiliar incluido (`upstream.conf`) dentro de un directorio montado (`conf.d`)?

3.2 Checklist de Logro Formativo (Autoevaluación)

Objetivo Formativo	Evidencia Observable	Estado
1. Orquestación Multi-Entorno	Contenedores Nginx, Blue (v1) y Green (v2) corriendo en red aislada.	[ ] Logrado
2. Archivo Auxiliar Upstream	Uso de `conf.d/upstream.conf` modular sin errores de inodo bloqueado.	[ ] Logrado
3. Smoke Test en Staging	Validación de endpoint `/api/health` en Green antes del switch.	[ ] Logrado
4. Zero Downtime Swap	Conmutación con `nginx -s reload` sin pérdida de paquetes HTTP.	[ ] Logrado
5. Rollback Instantáneo	Retorno a Blue en < 1 segundo mediante reversión de archivo auxiliar.	[ ] Logrado